

Trabalho Final: Compiladores

Construção do Compilador para a Linguagem Homi

Carlos Eduardo Velozo e Lucas Xavier Pairé

1 Introdução e Visão Geral

A linguagem **Homi** foi desenvolvida para simplificar a criação de automações no sistema *Home Assistant*. O compilador traduz scripts lógicos, com estrutura baseada em Gatilhos (QUANDO), Condições (SE) e Ações (ENTAO), em arquivos de configuração declarativos no formato YAML, exigidos nativamente pelo sistema alvo.

Abaixo é apresentado um script desenvolvido na linguagem Homi e o seu respectivo arquivo YAML gerado pelo compilador:

Script Homi Original:

```
AUTOMACAO "Rotina Noturna"
QUANDO horario 22:30
SE light.sala_estar estiver desligado
ENTAO ligar light.quarto
```

YAML Gerado pelo Compilador:

```
- id: '4581293041'
  alias: "Rotina Noturna"
  description: "Gerado automaticamente pelo Compilador Homi"
  triggers:
    - platform: time
      at: '22:30:00'
  conditions:
    - condition: state
      entity_id: light.sala_estar
      state: 'off'
  actions:
    - action: light.turn_on
      target:
        entity_id: light.quarto
  mode: single
```

2 Fase 1: Analisador Léxico

O analisador léxico foi implementado em Python utilizando a biblioteca de Expressões Regulares (re). O algoritmo varre o código fonte agrupando a entrada em *Tokens* estruturados e registrando o número da linha para a emissão de erros, enquanto caracteres de espaçamento e comentários são sumariamente ignorados.

Token	Expressão Regular (Padrão)	Exemplo de Lexema
AUTOMACAO	\bAUTOMACAO\b	AUTOMACAO
QUANDO	\bQUANDO\b	QUANDO
SE	\bSE\b	SE
ENTAO	\bENTAO\b	ENTAO
OP_LOGICO	\b(E OU)\b	E, OU
ACAO	\b(ligar desligar alternar notificar)\b	ligar, notificar
ESTADO	\b(ligado desligado aberto ... ocioso)\b	ligado, aberto
OPERADOR	\b(for estiver)\b	estiver
TIPO_GATILHO	\b(horario tempo)\b	horario
TEMPO_EXATO	\b\d{2}:\d{2}\b	22:30
TEMPO_UNIT	\b\d+(s m min h hs)\b	10s, 5min
ID_ENTIDADE	\b[a-z_]+\.[a-z0-9_]+\b	light.quarto
STRING	".*?"	"Rotina Noturna"
ESPACO	[\t]+	(Ignorado)
NOVA_LINHA	\n	(Ignorado)
COMENTARIO	#. *	# (Ignorado)
ERRO	.	(Qualquer outro caractere)

Tabela 1: Especificação completa de Tokens do Analisador Léxico

3 Fase 2: Analisador Sintático

O compilador utiliza um analisador sintático **Descendente Preditivo Tabular Não-Recurso** LL(1).

3.1 Gramática Livre de Contexto (GLC)

A gramática foi fatorada à esquerda e suas recursões foram eliminadas. Abaixo, os símbolos **Terminais** estão representados em formato monoespaçado (ex: QUANDO), enquanto os **Não-Terminais** estão em texto padrão (ex: BlocoQuando). O símbolo ε (Epsilon) representa derivações vazias.

1. Programa $\rightarrow$ AUTOMACAO STRING BlocoQuando BlocoSe BlocoEntao
2. BlocoQuando $\rightarrow$ QUANDO RegraGatilho
3. RegraGatilho $\rightarrow$ TIPO_GATILHO ComplementoGatilho
4. RegraGatilho $\rightarrow$ ID_ENTIDADE OPERADOR ESTADO
5. ComplementoGatilho $\rightarrow$ TEMPO_EXATO
6. ComplementoGatilho $\rightarrow$ TEMPO_UNIT
7. BlocoSe $\rightarrow$ SE RegraCondicao
8. BlocoSe $\rightarrow \varepsilon$
9. RegraCondicao $\rightarrow$ ID_ENTIDADE OPERADOR ESTADO MaisCondicoes
10. MaisCondicoes $\rightarrow$ OP_LOGICO RegraCondicao

11. MaisCondicoes $\rightarrow \varepsilon$
12. BlocoEntao $\rightarrow$ ENTAO Comando MaisComandos
13. Comando $\rightarrow$ ACAO Complemento
14. Complemento $\rightarrow$ ID_ENTIDADE
15. Complemento $\rightarrow$ STRING
16. MaisComandos $\rightarrow$ OP_LOGICO Comando MaisComandos
17. MaisComandos $\rightarrow \varepsilon$

3.2 Conjuntos FIRST e FOLLOW e Tabela Preditiva

Para comprovar o determinismo da tabela preditiva, os conjuntos FIRST e FOLLOW foram computados:

Não-Terminal	FIRST	FOLLOW
Programa	{AUTOMACAO}	{ \$ }
BlocoQuando	{QUANDO}	{SE, ENTAO}
RegraGatilho	{TIPO_GATILHO, ID_ENTIDADE}	{SE, ENTAO}
ComplementoGatilho	{TEMPO_EXATO, TEMPO_UNIT}	{SE, ENTAO}
BlocoSe	{SE, ε }	{ENTAO}
RegraCondicao	{ID_ENTIDADE}	{ENTAO}
MaisCondicoes	{OP_LOGICO, ε }	{ENTAO}
BlocoEntao	{ENTAO}	{ \$ }
Comando	{ACAO}	{OP_LOGICO, \$}
Complemento	{ID_ENTIDADE, STRING}	{OP_LOGICO, \$}
MaisComandos	{OP_LOGICO, ε }	{ \$ }

Tabela 2: Conjuntos FIRST e FOLLOW

A intersecção desses conjuntos preenche a Tabela Preditiva LL(1). Por se tratar de uma matriz esparsa com muitos terminais, a representação bidimensional (onde as linhas representam Não-Terminais e as colunas os Terminais) foi fracionada em três tabelas menores para assegurar a legibilidade das produções. Células vazias indicam transições inexistentes (erros sintáticos).

Não-Terminal	AUTOMACAO	QUANDO	TIPO_GATILHO	TEMPO_EXATO	TEMPO_UNIT
Programa	Prog $\rightarrow$ AUTOMACAO STRING BlocoQuando BlocoSe BlocoEntao				
BlocoQuando		BlocoQuando $\rightarrow$ QUANDO RegraGatilho			
RegraGatilho			RegraGatilho $\rightarrow$ TIPO_GATILHO CompGatilho		
ComplementoGatilho				CompGat $\rightarrow$ TEMPO_EXATO	CompGat $\rightarrow$ TEMPO_UNIT
BlocoSe					
RegraCondicao					
MaisCondicoes					
BlocoEntao					
Comando					
Complemento					
MaisComandos					

Tabela 3: Matriz LL(1) - Parte 1: Início e Gatilhos

Não-Terminal	ID_ENTIDADE	OPERADOR	ESTADO	STRING	ACAO
Programa					
BlocoQuando					
RegraGatilho	RegraGat $\rightarrow$ ID_ENTIDADE OPERADOR ESTADO				
ComplementoGatilho					
BlocoSe					
RegraCondicao	RegraCond $\rightarrow$ ID_ENTIDADE OPERADOR ESTADO MaisCond				
MaisCondicoes					
BlocoEntao					
Comando					Comando $\rightarrow$ ACAO Complemento
Complemento	Comp $\rightarrow$ ID_ENTIDADE			Comp $\rightarrow$ STRING	
MaisComandos					

Tabela 4: Matriz LL(1) - Parte 2: Entidades e Ações

Não-Terminal	SE	ENTAO	OP_LOGICO	EOF (\$)
Programa				
BlocoQuando				
RegraGatilho				
ComplementoGatilho				
BlocoSe	BlocoSe $\rightarrow$ SE RegraCondicao	BlocoSe $\rightarrow \epsilon$		
RegraCondicao				
MaisCondicoes		MaisCond $\rightarrow \epsilon$	MaisCond $\rightarrow$ OP_LOGICO RegraCondicao	
BlocoEntao		BlocoEntao $\rightarrow$ ENTAO Comando MaisComandos		
Comando				
Complemento				
MaisComandos			MaisCom $\rightarrow$ OP_LOGICO Comando MaisComandos	MaisCom $\rightarrow \epsilon$

Tabela 5: Matriz LL(1) - Parte 3: Condições e Fim de Pilha

3.3 Recuperação de Erros por Modo Pânico

Para conferir tolerância a falhas, implementou-se a estratégia de **Modo Pânico**. Ao identificar um token inválido, o analisador suspende a validação e descarta tokens contínua e sequencialmente até atingir um dos símbolos de sincronização principais: $\{\text{QUANDO, SE, ENTAO, EOF}\}$. Isso permite isolar a instrução corrompida e continuar analisando os blocos subsequentes, relatando múltiplos erros de uma vez.

4 Fase 3: Analisador Semântico

O analisador semântico foca em validar a consistência lógica das instruções frente ao domínio do *Home Assistant*.

4.1 Tabela de Símbolos

Foi implementada em memória uma Tabela de Símbolos responsável por registrar os identificadores de entidades e seus respectivos domínios. Para garantir a extensibilidade da linguagem e permitir o uso de novos dispositivos sem a necessidade de atualização prévia no compilador, a Tabela de Símbolos é populada de forma dinâmica. O analisador semântico extrai o prefixo do `ID_ENTIDADE` e o registra automaticamente na tabela como sendo o domínio correspondente daquele dispositivo.

Identificador da Entidade Lida (<code>ID_ENTIDADE</code>)	Domínio Inferido Dinamicamente
<code>light.sala_estar</code>	<code>light</code>
<code>switch.ilha_interruptor</code>	<code>switch</code>
<code>sensor.temperatura</code>	<code>sensor</code>
<code>binary_sensor.porta_entrada</code>	<code>binary_sensor</code>

Tabela 6: Exemplo de inserção dinâmica na Tabela de Símbolos

A partir do cruzamento dessas informações, duas restrições semânticas centrais foram implementadas:

- **Validação de Ações:** Impede comandos incompatíveis com a natureza do hardware (ex: enviar a ação `ligar` a um sensor, que atua exclusivamente como entidade de leitura).
- **Validação de Estados:** Rejeita checagens irreais baseadas no tipo mapeado da entidade. Por exemplo, uma entidade do tipo `light` aceita os estados `ligado` ou `desligado`, mas disparará um erro semântico se checada contra o estado `aberto`.

5 Fase 4: Geração de Código

Nesta etapa de síntese, a árvore sintática previamente validada é traduzida para a linguagem-alvo estruturada. O gerador de código converte a lógica declarativa em português para os blocos equivalentes nativos do Home Assistant no formato YAML.

Para suprir o vocabulário exigido pelo formato alvo, o compilador automatiza as seguintes conversões essenciais:

- **Injeção de Metadados:** Toda automação gerada recebe um `id` numérico único (aleatório com 10 dígitos) e a propriedade de execução mandatória `mode: single`.
- **Gatilhos (QUANDO):** Gatilhos baseados em relógio mapeiam para `platform: time`, com inserção automática de segundos (ex: `22:30` torna-se `22:30:00`). Gatilhos baseados em mudança de dispositivo geram um bloco `platform: state`.

- **Normalização de Estados:** Uma tabela *hash* embutida traduz estados em português para a base nativa (*inglês*). Termos dinâmicos como *ligado*, *aberto* e *movimento* são centralizados como *'on'*. Analogamente, *desligado*, *fechado* e *ocioso* geram *'off'*, enquanto *quente* vira *'hot'*, e *frio* vira *'cold'*.
- **Resolução de Ações Físicas (ENTAO):** O gerador traduz o verbo (*ligar* → *turn_on*, *desligar* → *turn_off*, *alternar* → *toggle*) e o concatena dinamicamente ao domínio da entidade afetada extraído na hora. Por exemplo, *ligar o alvo light.sala* gera a chamada algorítmica ao serviço *light.turn_on*.
- **Serviço de Notificações:** A ação nativa *notificar* desvia do fluxo de dispositivos padrão para gerar o serviço de sistema *notify.persistent_notification*, encapsulando a *STRING* lida pelo analisador como argumento da propriedade estrutural *message*.

Com os nós traduzidos, a síntese final consolida o documento YAML serializando os dados através de concatenação de blocos com indentação estrita de dois espaços. O arquivo emitido está imediatamente pronto para importação via interface do Home Assistant.

6 Exemplos Adicionais e Tratamento de Erros

Para demonstrar a robustez do compilador em todas as fases, foram elaborados cenários de teste projetados propositalmente para acionar os mecanismos de contenção de erros de cada analisador. Quando qualquer um desses cenários ocorre, a geração do YAML final é rigorosamente abortada.

6.1 Erro Léxico (Caracteres Inválidos)

O analisador léxico descarta os caracteres não reconhecidos sem abortar imediatamente a leitura (processando o restante da string), emitindo logs exatos da linha afetada.

Script de Entrada:

```
AUTOMACAO "Erro Léxico"
QUANDO horario 10:00
@invalido
ENTAO ligar light.sala_estar
```

Saída do Compilador:

```
[ERRO LÉXICO] Caractere ou palavra inválida '@' na linha 3
[ERRO LÉXICO] Caractere ou palavra inválida 'i' na linha 3
...
[ERRO LÉXICO] Caractere ou palavra inválida 'o' na linha 3
```

6.2 Erro Sintático (Estrutura Incompleta e Modo Pânico)

Neste teste, a omissão proposital da palavra-chave inicial **QUANDO** força um erro sintático. O **Modo Pânico** entra em ação, descartando tokens até localizar um ponto estrutural seguro para continuar a compilação e achar outros erros no arquivo.

Script de Entrada:

```
AUTOMACAO "Erro Sintático"
SE light.sala_estar estiver ligado
ENTAO ligar light.quarto
```

Saída do Compilador:

```
[ERRO SINTÁTICO] Falha ao derivar 'BlocoQuando' com token 'SE' na
linha 2.
[ERRO SINTÁTICO] Modo Pânico ativado na linha 2. Descartando tokens
até encontrar: ['QUANDO', 'SE', 'ENTAO', 'EOF']
```

6.3 Erro Semântico (Ação Incompatível com o Hardware)

A instrução tenta aplicar a ação `ligar` a uma entidade do domínio de leitura `binary_sensor`, violando as regras cruzadas na Tabela de Símbolos.

Script de Entrada:

```
AUTOMACAO "Erro Semântico Sensor"
QUANDO horario 10:00
ENTAO ligar binary_sensor.porta
```

Saída do Compilador:

```
[ERRO SEMÂNTICO] Incompatibilidade de Tipo: Não é possível 'ligar'
a entidade 'binary_sensor.porta' (Domínio: binary_sensor).
```

6.4 Erro Semântico (Estado Incompatível)

A instrução tenta verificar se uma iluminação (`light`) possui o estado `aberto`. Por não fazer sentido físico no domínio, o semântico intercepta a falha.

Script de Entrada:

```
AUTOMACAO "Erro Semântico Estado"
QUANDO horario 10:00
SE light.sala_estar estiver aberto
ENTAO ligar light.quarto
```

Saída do Compilador:

```
[ERRO SEMÂNTICO] Incompatibilidade de Tipo: A entidade 'light.
sala_estar' (light) não suporta o estado 'aberto'.
```

Esses cenários provam que a arquitetura do compilador não apenas traduz com sucesso casos ideais, mas também intercepta falhas clássicas de lógica e de gramática, prevenindo de forma absoluta a geração de YAMLS corrompidos.